



Part 3 of the Architecture Process

- ◆ Currently we know **what** we want to build.
- ◆ Now we deal with **how** we want to build it.
- ◆ First, we start with a first technical concept, the logical architecture. Here, we address technical properties of the later system.

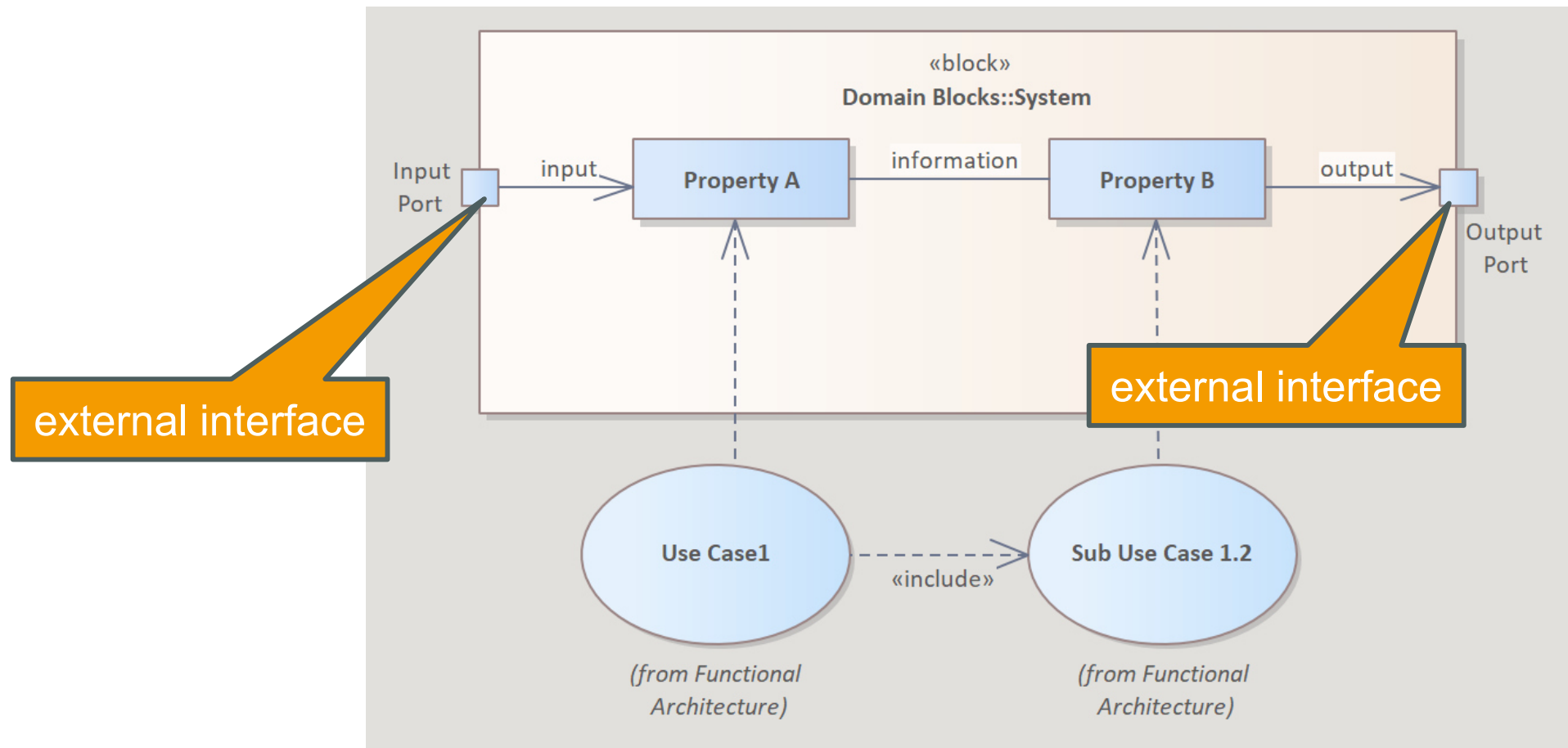
- ◆ Goal of the **logical** architecture is
 - Analysis of the system, so that physical elements can be realized
 - Identify the **external interface** of the system (**ibd**)
 - Identify the **internal (logical) interfaces** and inner connections between logical properties to derive meaningful interfaces in the physical architecture (**ibd**)
 - Refine functions having the logical elements in mind (**act**)

- ◆ The logical architecture is a step in between. It connects the functional architecture with the physical architecture



External Interfaces using Internal Block Diagram (ibd)

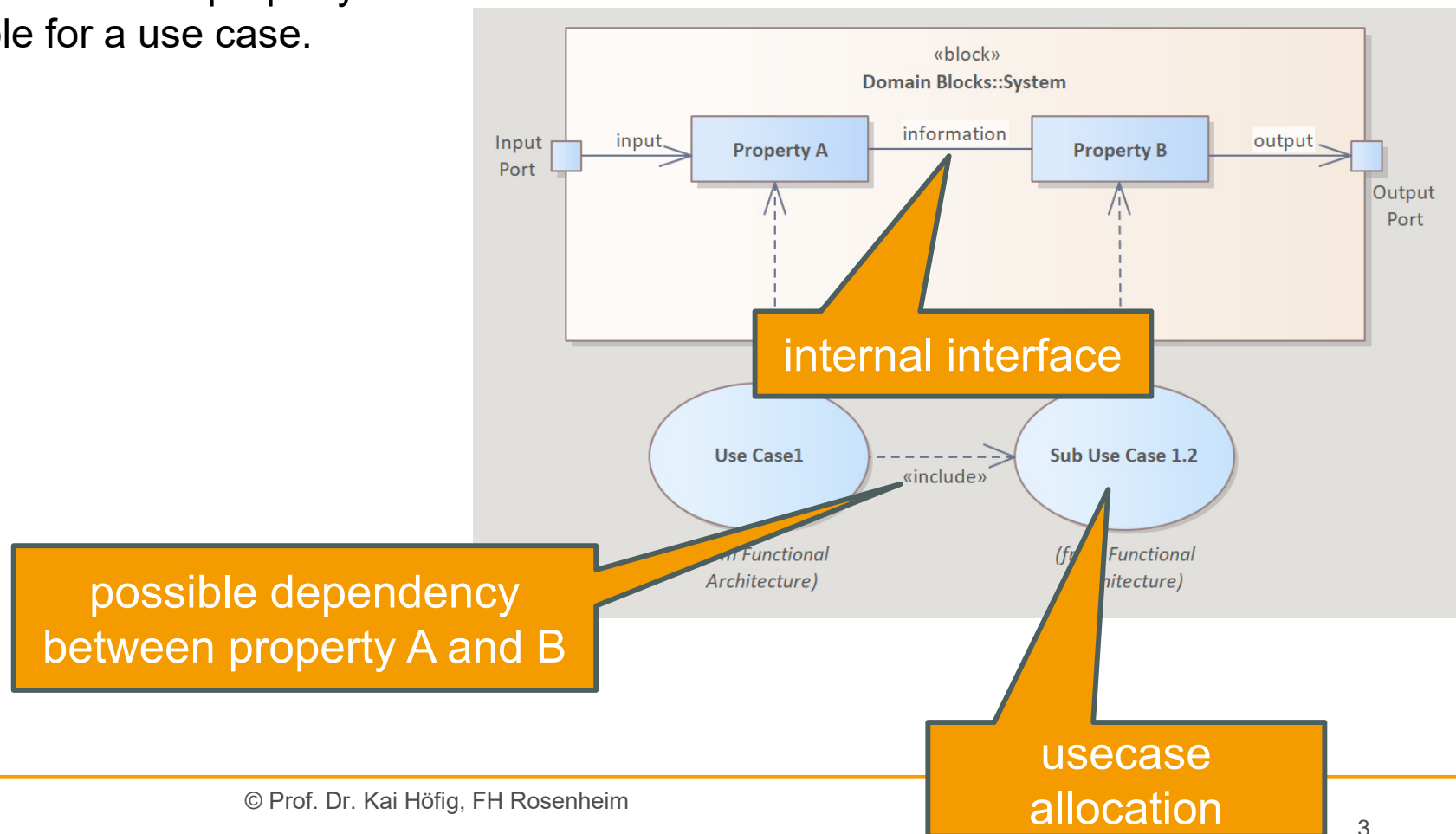
- ♦ The external interface of the system is modeled using ports. Derived from the use cases, these ports show which goods/information is coming from the outside or is provided to the outside of the system.





Logical Internal Interfaces using Internal Block Diagram (ibd)

- ♦ Logical internal interfaces show how information or flow between system properties is exchanged, having the use cases in mind.
- ♦ Those (sub)system blocks are modeled using properties to differentiate them from real physical blocks.
- ♦ Use cases show which property is responsible for a use case.





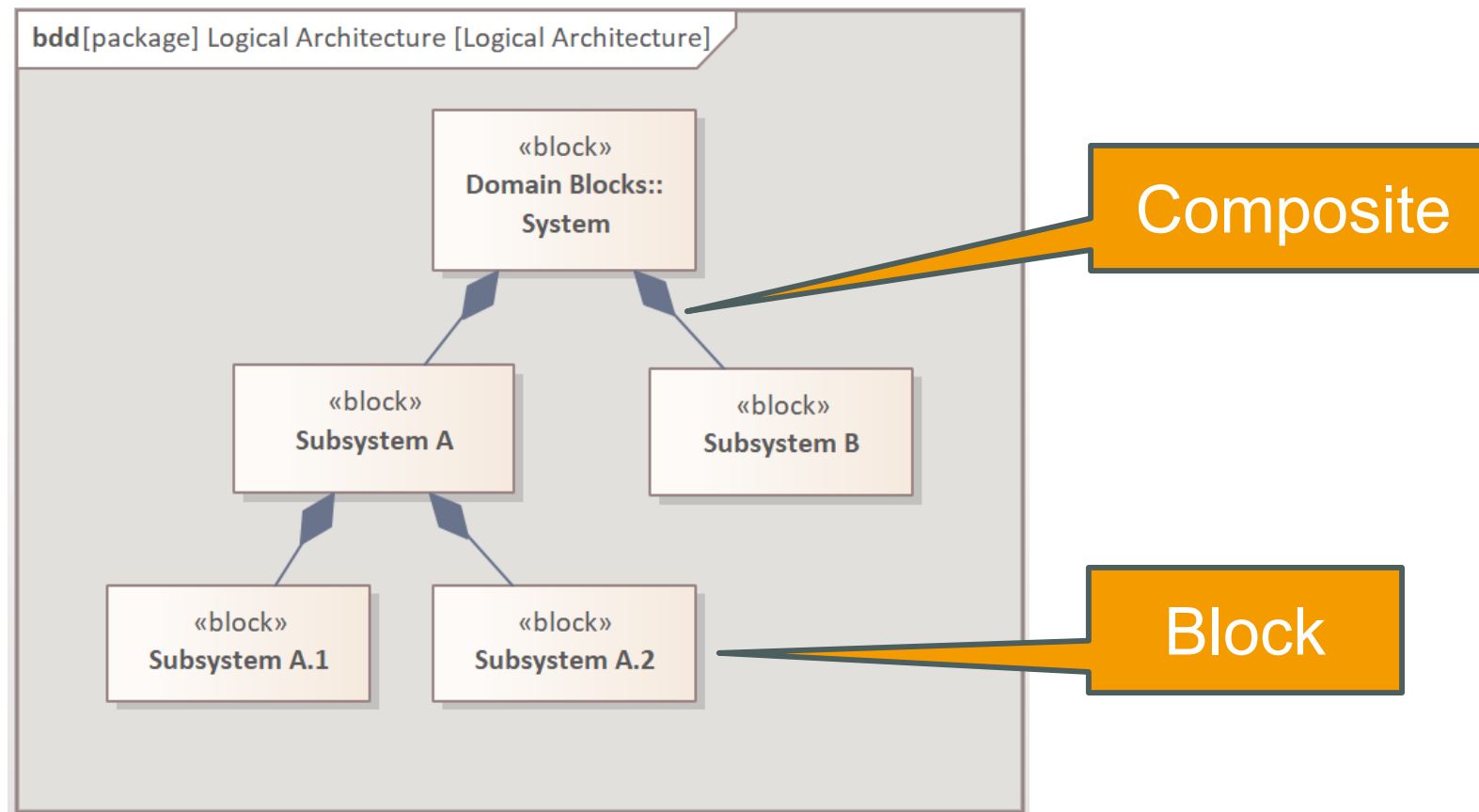
Part 4 of the Architecture Process

- ◆ Currently, we know the external interface and we have a conceptual idea of what the systems consist of. We also know how the functions will precisely be realized (design decisions).
- ◆ Goal of the **physical** architecture is to realize the system using the model so that it is clear how to build the system.
- ◆ Blocks are the physical subsystems
- ◆ Ports are physical external or internal interface elements
- ◆ Connections between ports are physical connections (power, goods, information)



Physical system decomposition using Block Definition Diagram (bdd)

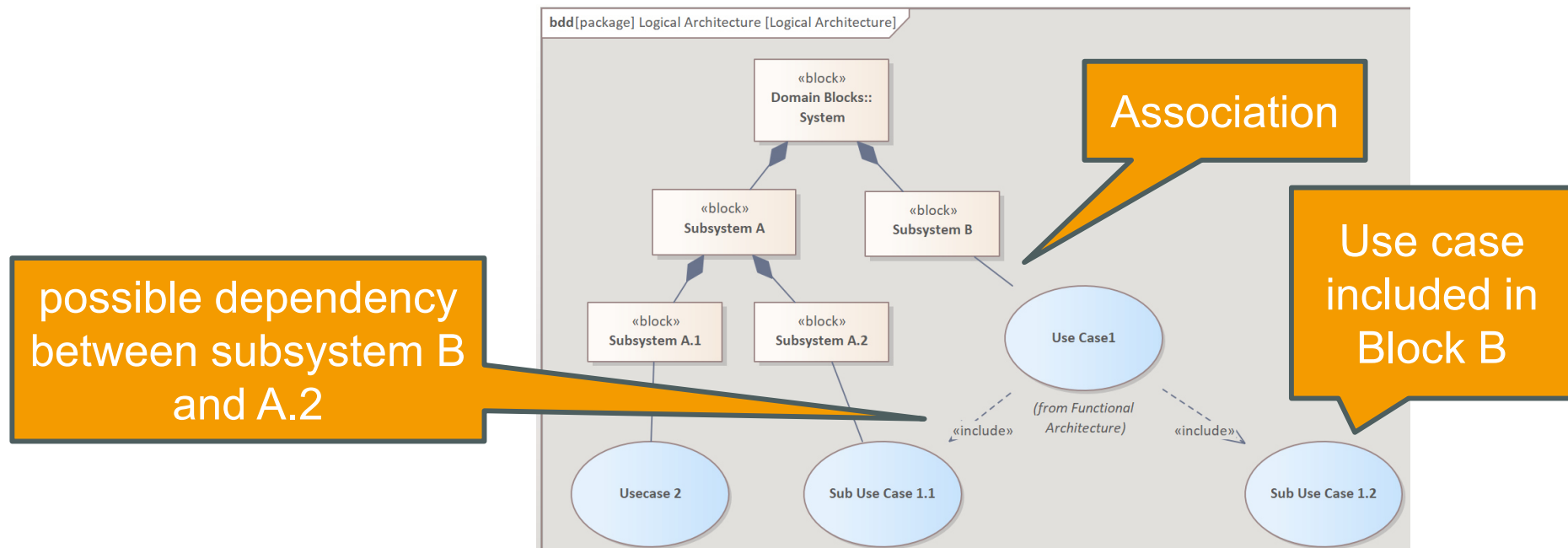
- ◆ We add a new **bdd** to the physical architecture.
- ◆ We add the system block using a **link** from the base architecture as root element.
- ◆ Subsystems are composites of the root element or systems. (set the compartment visibility!)





Usecase allocation in physical decomposition

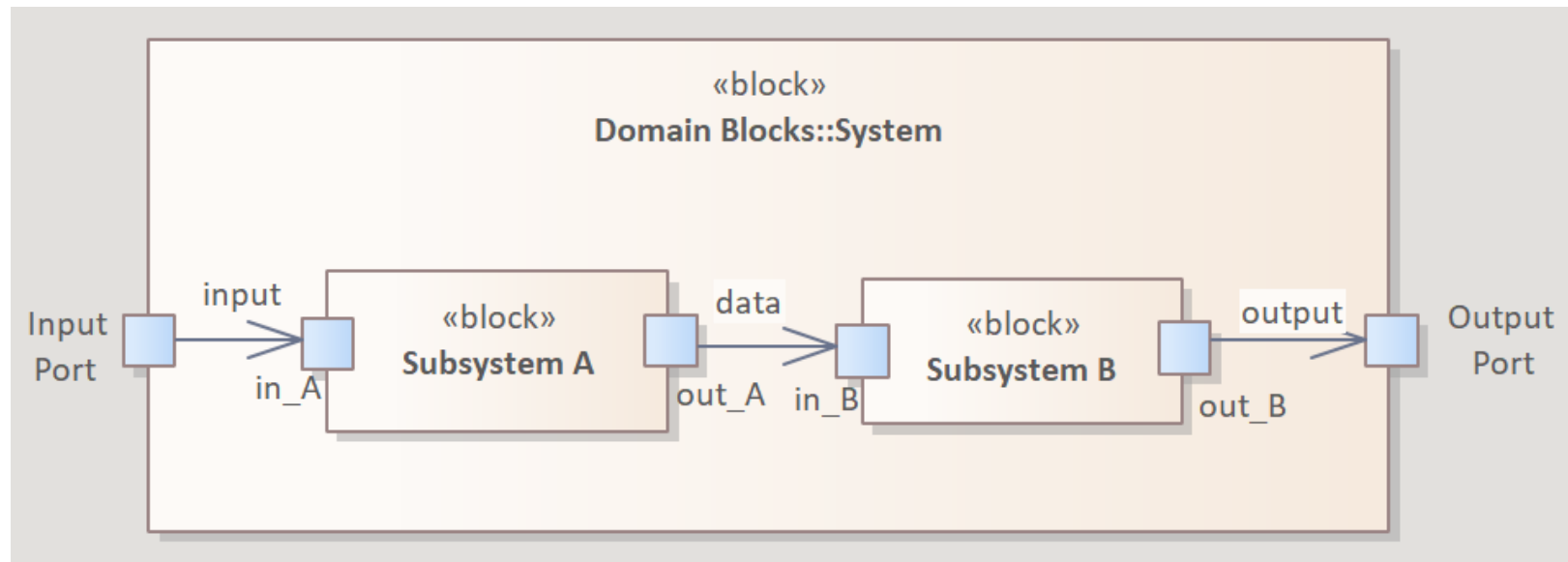
- ◆ In the **bdd** of the physical decomposition, the use cases from the base architecture are associated to the physical blocks by **links** using associations. Each use case has one block that is mainly responsible for the use case.
- ◆ Subsystems person responsible can trace to the activity diagram.
- ◆ Systems Engineer can see dependencies between subsystems.
- ◆ Systems Engineer can see whether all use cases are addressed





Physical implementation of static elements

- ♦ The physical blocks represent real physical elements.
- ♦ Each block is something to build, a property is not
- ♦ Each port is something to build, a connection without port is not





Done?

- ◆ Functional architecture is done when we know what to build and that we can do it.
- ◆ logical architecture is done when the external interface is clear, and we know how to make a good physical architecture-
- ◆ Physical architecture is done, when we know how to build the system.



Further Diagrams

- ◆ Each architecture step can contain different diagrams supporting the process. Besides Block Definition Diagram (**bdd**), Internal Block Diagram (**ibd**), Activity Diagram (**act**), Requirements Diagram (**req**) and Use Case Diagram (**uc**), SysML knows some more:
 - Package Diagram – Shows packages and dependencies
 - Parametric Diagram – Used to model parameter constraints
 - State Machine Diagram – more precise than the activity diagram
 - Sequence Diagram – Better to model interactions between elements than the activity diagram.



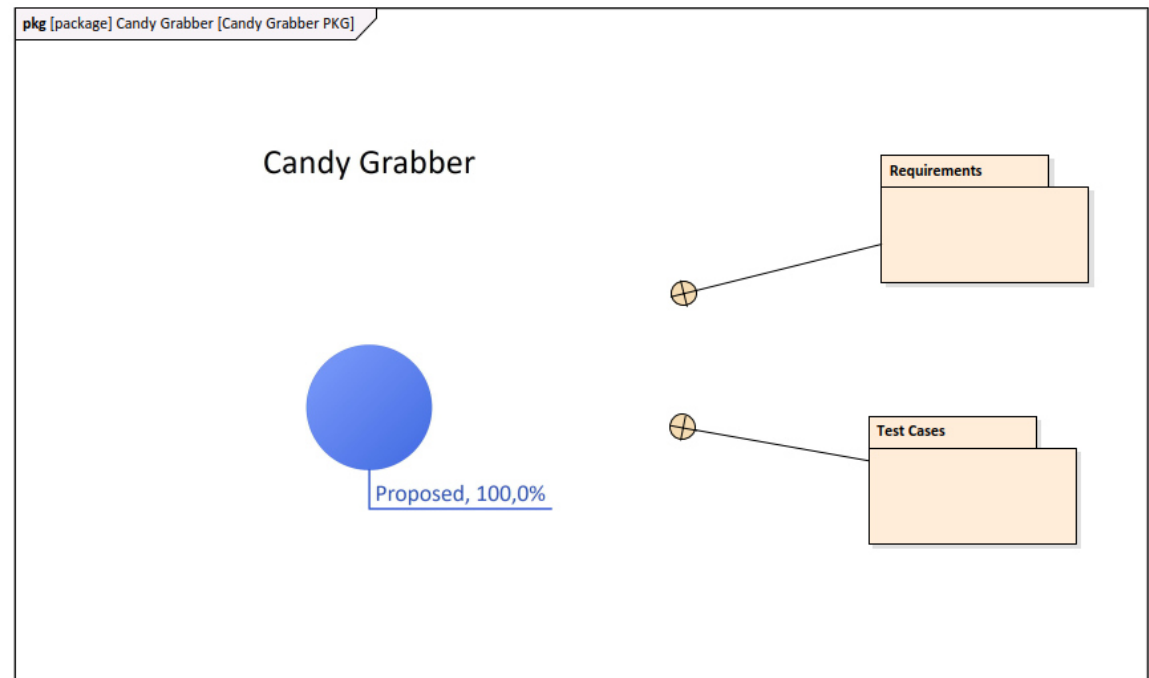
Package Diagram (pkg)

Central structural
modeling elements

- **Models**
Models are the root elements
- **Packages**
Packages are folders that can be used to structure the model elements.

Central
relationships

- **Connector**
Used to link packages
- **Containment**
Used to model a hierarchy



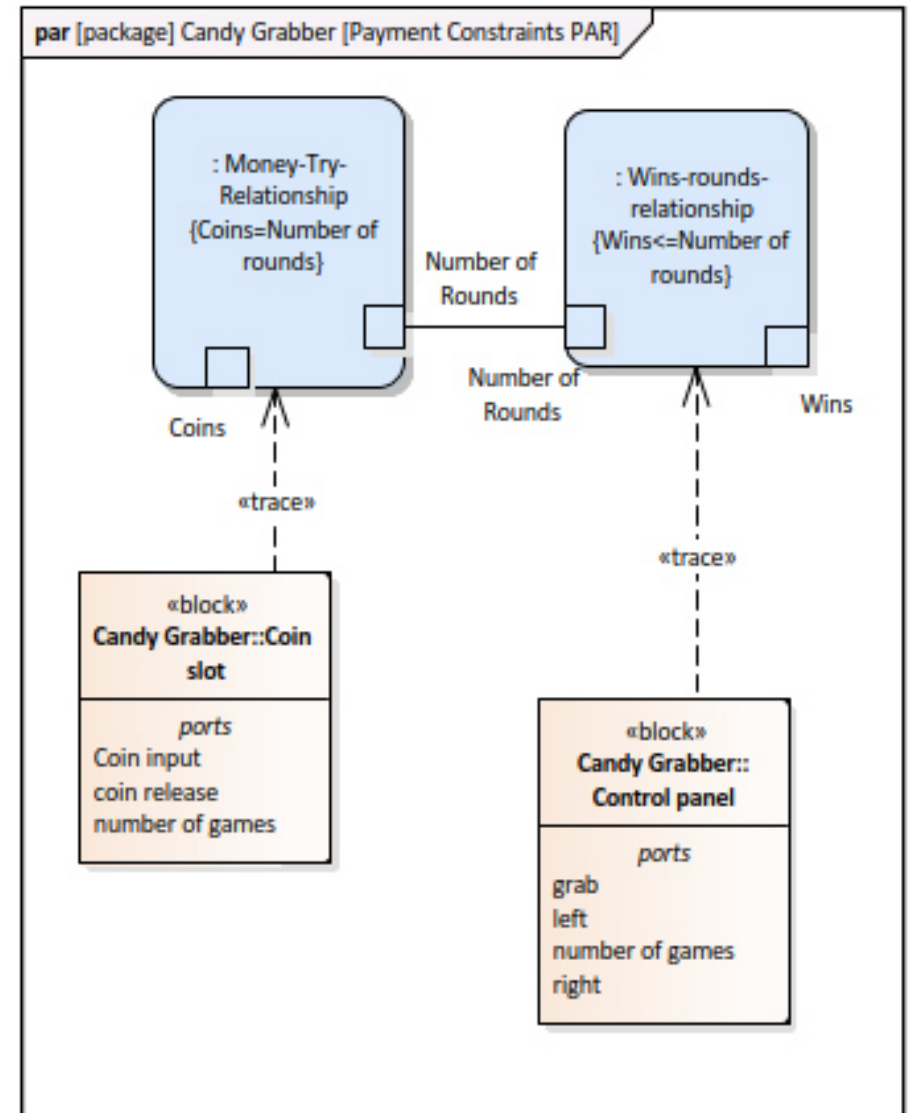
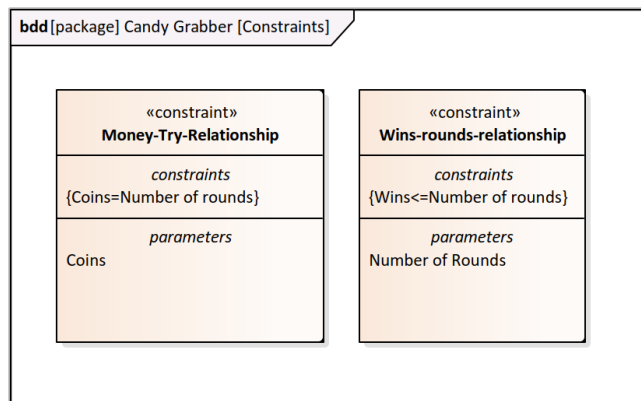


Parametric Diagram (par)

Central structural
modeling elements

Central
relationships

- **Constraints**
Required for a meaningful use of a parametric diagram. Constraints are modeled in a bdd.
- **Constraint Property**
Representation of constraints in parametric diagrams.
- **Property**
A parametric value
- **Connector**
Used to link properties with each other





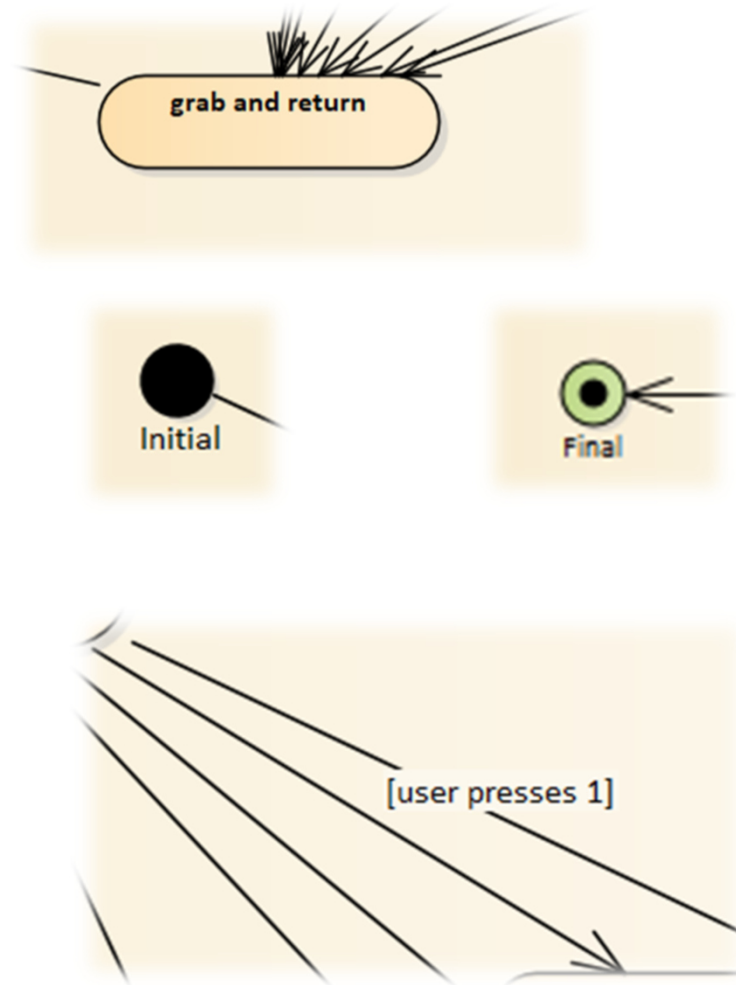
State Machine Diagram (stm)

Central structural
modeling elements

- **State Machine**
Wrapper element for a state machine.
- **State**
Element representing a system or subsystem state.
- **Start and End**
Source and sink element for the state machine.

Central
relationships

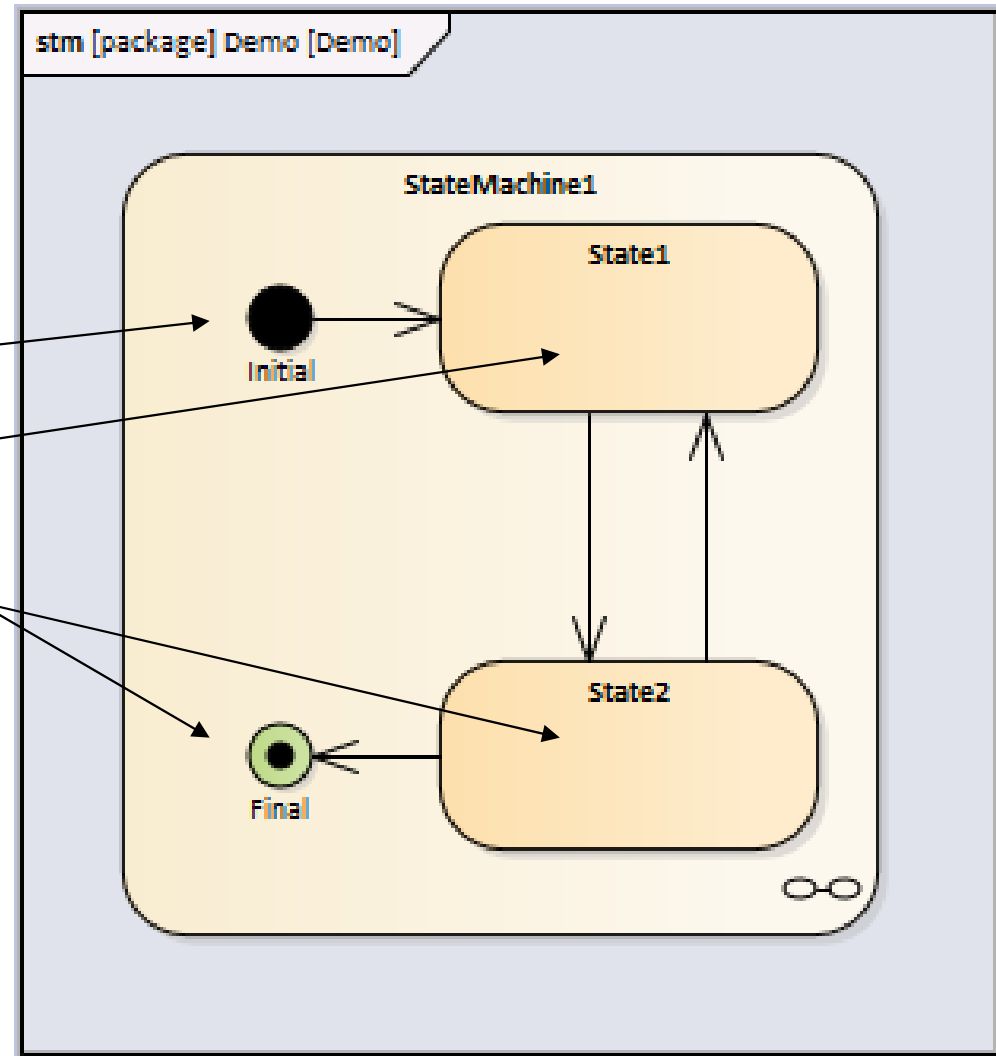
- **Transition with guard**
Transition from one state to another along with a guard condition when this transition is taken.





Example STM

- ◆ Initial and final state
- ◆ Internal states that can reach each other

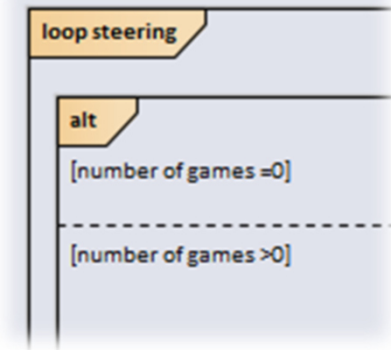




Sequence Diagram (sq)

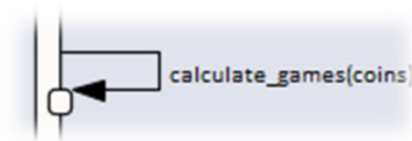
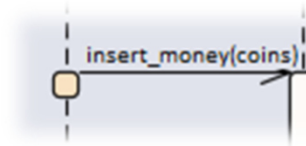
Central structural modeling elements

- **Lifeline**
Any classifier can get a lifeline to model sequences of interactions
- **Fragment**
Element to model more sophisticated interactions such as loops, alternatives or parallel executions



Central relationships

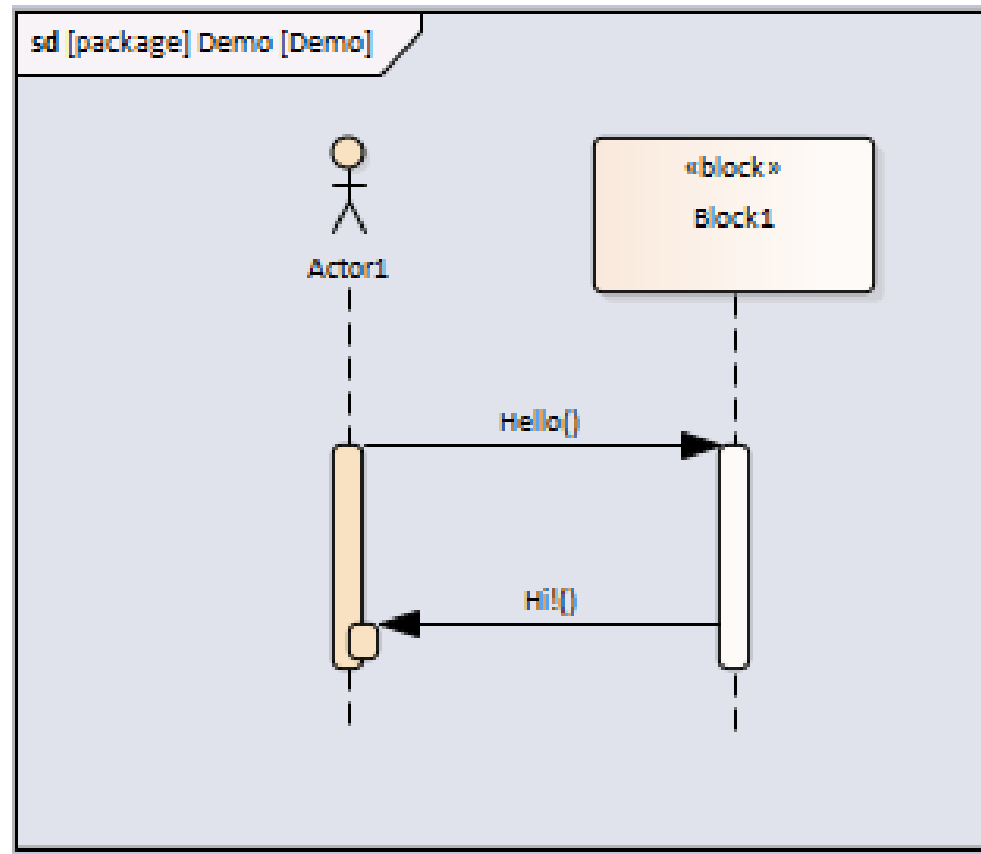
- **Synchronous message**
Sender is waiting for an answer
- **Asynchronous message**
Sender is not waiting for an answer (e.g. a signal)
- **Self-message**
Used to model internal calculations





Example SQ

- ◆ Actor1 send a hello message to Block1.
- ◆ Block1 calculates a bit and replies Hi !





- ◆ **Requirement Diagram** (req), Requirements and decomposition

- ◆ **Structural Diagrams**
 - **Block definition diagram** (bdd), static structure and decomposition of a system
 - **Internal block diagram** (ibd), internal view of a block
 - **Package diagram** (pkg), structure of modeling elements
 - **Parametric diagram** (par), parameter constraints in the system

- ◆ **Behavioral Diagrams**
 - **Activity diagram** (act)
 - **Use case diagram** (uc)
 - **State machine diagram** (stm)
 - **Sequence diagram** (sq)

SYSMOD Analysis Process



1. Define system idea and goals (req)
Description of basic principles, purpose of the system
2. Define base architecture (bdd), (ibd)
Define technological constraints and implications for the project.
3. Requirements engineering includes stakeholder analysis (uc), (req)
The stakeholder analysis provides the sources of requirements.
4. Modelling system context (interfaces of the system) (bdd), (ibd)
The system interacts at its **boundary** with external **actors** (people or systems) and exchanges **objects** via its **interface**
5. Define usecases that further describe functional requirements (uc)
Usecases are free of redundancies and precisely described using activity diagrams
6. Identify domain blocks (ibd), (bdd)
Domain blocks are things that the system is aware of. This step aims at summarizing those domain blocks along with their properties.